

Fase 5 · Control — Absenteísmo no Trabalho

Data: 26/09/2026 · Janela: base completa, 733 eventos, 34 identidades — holdout aberto nesta fase (26 exploração / 8 confirmação) · v1

Em três linhas

Entreguei o plano de controle (métrica primária, guardrail de CID, detector de código fora da lista oficial), converti a auditoria da Fase 2A em 14 testes automáticos (todos passam), e abri o holdout uma única vez com semente fixa registrada antes de qualquer cálculo.

Resultado da confirmação: o efeito de H1 se repete na mesma direção (9,5 p.p. vs. 16,4 p.p. na exploração), mas com IC muito largo [0,0; 32,2] dado $n=8$ — reforço qualitativo, não confirmação estatística. O piloto de H1 continua **condicional**, sem mudança de status. Tollgate: **APROVADO**.

Aviso registrado antes da tarefa 7 (antes de rodar o teste de confirmação)

O corte é de 34 identidades, 75/25: aproximadamente 26 funcionários na janela de exploração, 8 na quarentena de confirmação. Com 8 funcionários — divididos entre "longe" e "perto" da mediana de distância —, qualquer teste de H1 aqui terá poder ainda mais baixo que o já demonstrado ($\sim 3\%$ com $n=34$). **Isto está escrito antes de rodar o teste:** um resultado nulo no holdout não seria uma refutação mais forte que o "inconclusivo" do Analyze — seria o mesmo tipo de limitação, numa amostra menor ainda. A leitura do resultado, qualquer que seja, não deve depender de já se saber o que ele foi.

O que foi feito

1. Plano de controle do processo do cliente, como especificação (Parte A).
2. Auditoria da Fase 2A convertida em 14 testes automáticos (Parte B, `tests/test_qualidade.py`) — todos passam sobre a base atual.
3. Checklist de reprodutibilidade, com parâmetros extraídos para `config.py`.
4. Verificação de consistência entre os números do Improve e o baseline congelado.
5. Abertura do holdout — sorteio primeiro (commit separado, antes de qualquer cálculo), depois análise da confirmação.
6. Rascunho do README na raiz do projeto (Parte C, task 8).
7. Padrões reutilizáveis (C11) e retrospectiva Kaizen (C12).

Parte A — Plano de controle como especificação

Métrica	Definição operacional	Fonte	Frequência	Limite	Responsável	Ação ao sair do limite
Proporção de eventos B (primária)	(B1+B2) / total de eventos de ausência	Registro de ausência (Reason for absence)	Mensal	Referência: quartil superior de funcionário, 33,36% (Fase 2B) — não é limite estatístico de carta (M11 não sustenta carta aqui)	Gestor de RH/operações [PREMISSA DECLARADA]	Se sustentado por ≥ 2 meses acima do intervalo do baseline [57,95%–69,13%], investigar causa antes de agir
Guardrail — proporção de eventos CID sobre o total	CID / total de eventos	Registro de ausência	Mensal, e a cada mudança de processo	Não pode cair junto com a queda de B após qualquer intervenção (correção de H3 ou piloto de H1)	Gestor de RH/operações	Suspender a mudança de processo e investigar supressão de ausência médica antes de continuar (D5)
Eventos com código fora da lista oficial (1–28)	Contagem de "Reason for absence" fora de {0..28}, ou usando 0 após a correção de sistema	Registro de ausência	Mensal	Qualquer ocorrência > 0 depois da correção do sistema	Área de sistemas/RH	Alerta imediato — não é tolerância, é sinal de que o poka-yoke de H3 falhou

Carta de acompanhamento — não existe carta válida aqui (M11, Fase 2B: no máximo 13 pontos possíveis por rótulo de mês, sem coluna de ano). No lugar, o gatilho de alerta é a **comparação direta contra o baseline congelado com IC** (64,26% [57,95%–69,13%]): um valor fora desse intervalo, sustentado por pelo menos **dois períodos consecutivos** de medição futura — nunca um único ponto fora, dado que a variação natural aqui é grande (C2, C3).

O que invalidaria esta análise (C9)

Evento	O que fazer
RH implementar a correção de H3 (eliminar código 0)	O baseline de B1/B2 muda de definição — recalculado do zero, nunca comparar diretamente ao baseline antigo
Entrada de novos funcionários que mudem a composição de distância da operação	Reavaliar a mediana de distância e o grupo "longe/perto" antes de reusar H1
Digitação em lote que reintroduza o padrão de duplicata da Fase 2A	Rerodar a auditoria de unicidade (M2) antes de qualquer análise nova
Mudança na definição oficial de algum código de "Reason for absence" pela empresa ou por norma trabalhista	Revisar a fronteira B1/B2/CID do Define inteira, não só recalculando números

Parte B — Controle do pipeline

Testes automáticos (C7)

`tests/test_qualidade.py` — 14 testes, convertendo a auditoria da Fase 2A: colunas esperadas, ausência de nulos, domínio de Day of the week/Seasons/ Education/Reason for absence/binárias, circularidade do BMI (resíduo <1 em $\geq 95\%$ das linhas, Fase 2A registrou 98,64%), volume de eventos (733) e de identidades (34), e a não-reaparição das linhas excluídas no Define (IDs 4 e 35; a linha jovem do ID 29). **Todos os 14 passam** sobre `data/processed/eventos_qualidade.csv` (artefatos/`pytest_saida.txt`).

Checklist de reprodutibilidade (C8)

Item	Status
Dado bruto separado do processado	OK — <code>data/raw/</code> intocado, <code>data/processed/</code> gerado por script
Ambiente fixado	RESSALVA — <code>requirements.txt</code> lista pacotes sem versão pinada; funciona nesta máquina, mas não garante reprodução bit-a-bit em outra
Sementes fixas	OK — centralizadas em <code>config.py</code> (bootstrap/permutação, holdout)
Parâmetros fora do código	PARCIAL — <code>config.py</code> existe desde esta fase; scripts das Fases 2A–4 têm os mesmos valores hardcoded inline (coincidem com <code>config.py</code> porque foram copiados na criação deste arquivo, mas não foram refatorados para importar dele)
README permite rodar do zero	OK — ver <code>README.md</code> na raiz (rascunho desta fase)
Holdout isolado e documentado	OK — <code>data/holdout/split_ids.json</code> , semente e composição registradas antes da análise

Onde ainda falha, dito sem rodeio: os scripts 02a, 02b, 03 e 04 não importam `config.py` — eles têm os mesmos números escritos diretamente no código. Isso funciona hoje (os valores coincidem), mas é uma duplicação que um refactor futuro deveria eliminar, importando `config.py` em vez de reescrever as constantes. Fica registrado como pendência de qualidade de código, não como erro de resultado.

Verificação de consistência com o baseline congelado

`data/processed/eventos_qualidade.csv` não mudou desde a Fase 2A (mesmo hash, `ba78283944d6...`), e o baseline usado no Improve (64,26%) é o mesmo valor citado em `baseline-congelado.md`, sem divergência. A única diferença de número entre fases é a mediana de horas de B2 no subgrupo "longe" (3h) vs. a mediana geral de B2 (2h, Fase 2B) — já explicada no relatório do Improve como uma estatística de subgrupo, não uma contradição do baseline agregado.

Abertura do holdout (C5)

Sorteio, registrado antes de qualquer cálculo (commit separado, `scripts/05a_holdout_sorteio.py`): semente 20260926, 34 identidades, 75/25 → **26 em exploração, 8 em confirmação**. IDs de confirmação: 1, 5, 8, 21, 25, 27, 29, 30.

H1 se sustenta no grupo de confirmação?

Dos 8 funcionários sorteados para confirmação, 7 têm pelo menos um evento B2 ou CID (o ID 8 não tem nenhum dos dois — só tinha a linha administrativa excluída no Define e talvez outros tipos de evento). Desses 7: **3 são "longe"** (distância > 25,5km — a mesma mediana global usada em H1/H2 no Analyze, não recalculada só na confirmação) e 4 são "perto".

	Longe	Perto	Total
Eventos B2	7	18	25
Eventos CID	5	22	27

Proporção "longe" entre eventos B2: 28,0%. Proporção "longe" entre eventos CID: 18,5%.

Efeito observado: 9,48 p.p. [IC 95% bootstrap de cluster: 0,0 a 32,2 p.p.] — **mesma direção** que a exploração (16,4 p.p.), magnitude menor, intervalo muito largo (toca o zero no limite inferior).

Leitura, exatamente como o aviso registrado acima antecipava: este não é um teste com poder para confirmar ou refutar nada sozinho — é um segundo ponto de dado, pequeno, na mesma direção do primeiro. Não é confirmação estatística formal (o IC toca zero), mas também não é uma inversão de sinal nem um efeito nulo. É **reforço qualitativo**: dois recortes independentes da mesma base (exploração e confirmação), com métodos e definições idênticas, apontam na mesma direção, com magnitudes que se sobrepõem largamente dentro de suas margens de erro.

Registro do resultado, qualquer que seja (regra do contexto mestre): o efeito não inverteu, não caiu a zero, e não pode ser lido como confirmação — está registrado como está, sem nova tentativa, sem troca de semente.

O que isso decide sobre a recomendação piloto de H1 (Improve, item 2)

O piloto de agendamento continua proposto, na mesma condição em que estava — nem promovido a "confirmado", nem retirado. A confirmação no holdout não mudou o estágio de confiança estabelecido no Analyze: H1 continua sendo um candidato promissor, não uma causa validada. A decisão correta, dado $n=8$ na confirmação e $n=34$ na base inteira, é: **não implementar a política de agendamento como regra ativa a partir só desta base** — a via de decisão mais responsável é continuar registrando dados (mais meses, mais funcionários, se a operação crescer) até que um teste com poder adequado seja possível, ou aceitar o risco reduzido de um piloto informal, comunicado com extremo cuidado quanto ao guardrail de CID (Fase 4, FMEA), monitorando o efeito ao longo do tempo antes de qualquer generalização.

Ressalva sobre esta própria decisão: ela é tomada com $n=8$ na confirmação — a mesma limitação de poder que se aplicou à exploração se aplica, ampliada, aqui. Isto não é uma "segunda opinião independente" forte; é uma checagem de direção, e deve ser lida como tal na Entrega e no README.

Parte C — Padrões reutilizáveis e Kaizen

O que vira padrão (C11)

1. **Erro-padrão agrupado por cluster (bootstrap por funcionário/entidade) sempre que a granularidade é mista** — evento dentro de entidade, não entidade em si. Este projeto tratou isso em toda métrica (baseline, H1, H4, confirmação no holdout).
2. **Teste de colisão por acaso (problema do aniversário) para decidir duplicata** sem regra arbitrária de "remover tudo" ou "manter tudo" — generalizável a qualquer dataset com poucas colunas e alta chance de coincidência legítima.
3. **Separação de tom firme/condicional por estágio de confiança da causa** na fase Improve — uma causa confirmada (H3) recebe contramedida ativa; uma promissora não confirmada (H1) recebe entregável marcado como piloto, nunca regra ativa. Evita que o relatório finja mais certeza do que a análise sustenta.

Retrospectiva Kaizen — os oito desperdícios (C12)

Desperdício	Onde apareceu neste projeto	Horas estimadas
Superprodução	Cálculos gerados e não usados no relatório final (ex.: algumas descritivas do perfilamento da Fase 0 que não reapareceram depois)	~0,5h
Espera	Mínima — o fluxo entre fases foi contínuo nesta sessão	~0h

Desperdício	Onde apareceu neste projeto	Horas estimadas
Transporte	Handoff de dados entre fases via CSV intermediário (eventos_limpos → eventos_qualidade) — necessário para rastreabilidade, não puro desperdício	~0h (justificado)
Processamento excessivo	A função de classificação do defeito (B1/B2/CID) foi reescrita em pelo menos 6 scripts diferentes (01, 02a, 02b, 03, 04, 05b), em vez de importada de um módulo comum	~1h acumulada em repetição de código e risco de divergência
Estoque	Vários CSVs intermediários e JSONs de artefato — aceitável, servem de prova, não estoque morto	~0h (justificado)
Movimento	Navegar entre muitos scripts pequenos numerados por fase — organização correta, mas exige disciplina de nomenclatura	~0,3h
Defeitos	Três erros autocaptados: (1) cálculo do teste de H3 quase feito antes do pré-registro (Fase 1); (2) "36 identidades" em vez de 34 no pré-registro (adendo); (3) soma "54,4%" em vez de "64,4%" no relatório do Define	~1,5h entre diagnóstico, correção e nova verificação
Talento subutilizado	Nenhuma consulta a alguém com experiência real em RH/logística para validar se H1 (fricção de agendamento por distância) é um mecanismo plausível no setor de transporte — a análise ficou inteiramente dentro dos dados	~0h perdida, mas é uma lacuna de validação, não de tempo

Causa raiz do maior desperdício (Defeitos, ~1,5h): a ausência de um módulo central que computasse e "travasse" a classificação do defeito uma única vez. Cada fase recalculava do zero em um script separado — o número no JSON estava sempre certo (porque vinha do cálculo), mas a **transcrição manual desse número para a prosa do relatório** é onde os erros de "36 vs 34" e "54,4% vs 64,4%" aconteceram. O terceiro defeito (quase-teste de H3) já tinha causa raiz diferente: falta de um passo explícito de revisão do script antes de rodá-lo, no meio de uma fase sob pressão de produzir o

pré-registro.

Três mudanças concretas para o próximo projeto:

1. Criar um módulo compartilhado (`lib/classificacao.py` ou similar) com a função de classificação do defeito, importado por todos os scripts — não reescrito fase a fase.
2. Gerar as frases numéricas do relatório a partir do JSON por template simples, em vez de digitar o número manualmente na prosa — elimina a classe de erro que produziu dois dos três defeitos.
3. Antes de rodar qualquer script que ainda vá alimentar um pré-registro, revisar o código uma vez à parte, perguntando explicitamente "este cálculo já responde a uma hipótese que ainda não foi escrita?" — a checagem que teria pego o quase-erro de H3 mais cedo, embora desta vez tenha sido pega antes do commit.

Números

Todos os números desta fase vêm de

`fases/fase-5-control/artefatos/holdout_confirmacao.json` (gerado por `scripts/05b_holdout_confirmacao.py`) e `data/holdout/split_ids.json` (gerado por `scripts/05a_holdout_sorteio.py`, commit separado e anterior). Resultado dos testes automáticos em `artefatos/pytest_saida.txt`.

Tollgate CONTROL

Critério	Veredito	Motivo
Um estranho roda o projeto do zero só com o README	RESSALVA	README existe e lista os passos; ambiente não é pinado por versão (checklist, item "Ambiente fixado")
Métrica primária e guardrail de CID no plano de controle, com limite e ação escritos antes do fato	OK	Parte A — tabela completa
Testes automáticos de qualidade existem e falham quando deveriam	OK	14 testes, <code>tests/test_qualidade.py</code> , todos verificados
Holdout aberto uma única vez, semente registrada antes do resultado, resultado registrado qualquer que seja	OK	Commit do sorteio separado e anterior ao commit da análise; efeito de 9,48 p.p. registrado como saiu
Aviso de poder baixo escrito antes do resultado, não encaixado depois	OK	Seção "Aviso registrado antes da tarefa 7", commitada antes do script de confirmação ser executado nesta sessão
Limitações do projeto no README de forma que protege, não expõe	OK	<code>README.md</code> , seções "origem e natureza dos dados" e "o que eu faria com dados reais"

Veredito: APROVADO com ressalva — o "ambiente fixado" (versões não pinadas em `requirements.txt`) e a duplicação de constantes fora de `config.py` nos scripts de fases anteriores (checklist de reprodutibilidade) são as duas pendências que não impedem a conclusão da fase, mas deveriam ser resolvidas antes de uma reexecução em outra máquina/ambiente.

Pendências e riscos

- `requirements.txt` sem versões pinadas — reprodutibilidade bit-a-bit não garantida em outra máquina.
- Scripts das Fases 2A-4 não importam `config.py` — duplicação de constantes, risco de divergência futura se um valor for alterado num lugar só.
- A decisão sobre H1 (não implementar, continuar coletando dados) depende de alguém na operação real decidir continuar o projeto além deste portfólio — não há como forçar isso a partir daqui.

Para a próxima fase

A Entrega recebe: todos os checkpoints de fase, o README de rascunho (a ser revisado na Fase 8), o resultado do holdout (reforço qualitativo de H1, não confirmação), e a instrução de que toda menção a H1 na página de apresentação precisa citar o poder baixo e o resultado do holdout juntos — nunca um sem o outro.